Contents

1 Motivation and Problem Context **3**

2 Reinforcement Learning Formulation **3**

3 Environment Description **3**
- 3.1 Environment . . . 3
- 3.2 State Representation . . . 3
- 3.3 Action Space . . . 3
- 3.4 Reward Design . . . 3

4 Environment Extensions **4**

5 Agent 1: Q-learning **4**
- 5.1 The Q-Table . . . 4
- 5.2 State Representation and Feature Extraction . . . 4
- 5.3 Policy: Epsilon-Greedy . . . 4
- 5.4 Learning Algorithm . . . 5
- 5.5 Challenges and Solutions . . . 5
- 5.6 Results and Visual Analysis (Q-Learning) . . . 5

6 Agent 2: PPO (Proximal Policy Optimization) **8**
- 6.1 Agent Workflow . . . 8
- 6.2 Training Evolution . . . 8
- 6.3 Visual Observations and Run Comparison . . . 9
- 6.4 Hyperparameter Analysis . . . 11

7 Agent 3: Deep Q-Network (DQN) **11**
- 7.1 Neural Function Approximation . . . 11
- 7.2 State Preprocessing and Input Representation . . . 11
- 7.3 Action Space . . . 11
- 7.4 Network Architecture . . . 11
- 7.5 Experience Replay and Target Network . . . 12
- 7.6 Optimization Objective . . . 12
- 7.7 Results and Visual Analysis (DQN) . . . 12
- 7.8 Other Observations and Discussion (DQN) . . . 13
- 7.9 Partial Conclusions (DQN) . . . 14

8 Experimental Methodology **14**
- 8.1 Aggregated Metrics Across Experiments . . . 14

9 Overall Comparison and Discussion **16**
- 9.1 PPO vs. Q-learning (Empirical Comparison) . . . 16
- 9.2 Discussion of Trade-offs . . . 17

10 Conclusion **17**

Reinforcement Learning for F1-Inspired Racing Strategy

A Comparative Study of Q-Learning, PPO, and DQN in `CarRacing`

Team Members: Andrei Stefan, Bejan Stefan, Cernea Ana, Navodaru Denisa, Stan David

January 15, 2026

1 Motivation and Problem Context

Formula 1 racing has evolved into a data-intensive discipline, where strategy, vehicle setup, and driver behavior are increasingly optimized using computational methods. Such optimizations can be extremely valuable for modern teams, especially **Ferrari**, whose recent seasons have highlighted the importance of strategic decision-making.

This project explores how **Reinforcement Learning (RL)** can contribute to racing strategy development through autonomous driving agents. We focus on a simplified racing task that captures key challenges:

- operating under uncertain and stochastic dynamics,
- balancing speed against stability and safety,
- learning effective control policies through trial and error.

Although the simulator does not model real Formula 1 physics, the learning principles and evaluation methodology are directly transferable to real-world autonomous racing problems.

2 Reinforcement Learning Formulation

The autonomous driving task is naturally formulated as a **Markov Decision Process (MDP)**, since the agent interacts sequentially with the environment and each action influences future states and rewards. The problem is defined by the tuple (S, A, R, γ):

- **State space (S):** Stacked visual observations (consecutive frames) describing the current driving situation. This representation satisfies the Markov property, as it captures all information required for decision-making.
- **Action space (A):** Control commands available to the agent. Continuous actions are discretized for value-based agents to enable Q-learning and DQN.
- **Reward function (R):** Encodes the driving objective by rewarding forward progress and penalizing off-track or unstable behavior.
- **Discount factor (γ):** Controls the importance of long-term rewards. A value close to 1 emphasizes planning over multiple timesteps, which is critical in racing scenarios.

3 Environment Description

3.1 Environment

Experiments are conducted in the `CarRacing-v2` environment, which simulates a vehicle driving on a closed racing circuit. The track is procedurally generated, introducing randomness between episodes.

3.2 State Representation

The environment provides image-based observations. To capture temporal information such as velocity and steering dynamics, the state is constructed by stacking $n = 4$ consecutive frames.

3.3 Action Space

The simulator expects continuous control signals. For value-based methods, the action space is discretized using a predefined action set, mapping discrete actions to continuous control vectors.

3.4 Reward Design

The baseline reward is taken from the environment. Additional reward shaping is explored to penalize off-track behavior and encourage smoother and faster driving.

4 Environment Extensions

Although `CarRacing-v2` is a standard benchmark, it is extended using custom wrappers:

- discrete action mapping,
- frame stacking,
- configurable episode termination conditions,
- YAML-based configuration files for reproducibility.

These extensions allow controlled experiments and systematic ablation studies.

5 Agent 1: Q-learning

The agent is built upon a tabular Q-learning framework. Below are the specific technical components governing its behavior.

5.1 The Q-Table

The core component of the agent is the **Q-table**, which stores the estimated action-value function.

- **Structure:** Implemented as a Python dictionary.
- **Mapping:** Each entry maps a state representation to a NumPy array.
- **Function:** For a given state key, the dictionary returns an array containing Q-values for all possible actions.
- **Initialization:** When the agent encounters a previously unseen state, the corresponding Q-values are initialized to zero.

5.2 State Representation and Feature Extraction

The environment provides high-dimensional pixel-based observations, which makes a direct tabular Q-learning approach computationally infeasible due to the infinite size of the state space.

To address this limitation, a **feature extraction** mechanism is employed.

- **Solution:** A feature extractor is used to reduce the dimensionality of the raw observations.
- **Method (Tile Coding):** The implementation adopts a `tile_v1` approach, which discretizes the continuous observation space into a finite grid of tiles.
- **Example:** Instead of tracking every continuous value of the car's position, the track is divided into an 8×8 grid. The state is represented by the indices of the tiles currently occupied by the vehicle.
- **Configurability:** The grid resolution (number of tiles) is a tunable hyperparameter, allowing a trade-off between state precision and memory usage.

5.3 Policy: Epsilon-Greedy

Action selection is governed by an **epsilon-greedy policy**, which balances exploration and exploitation.

- **Exploration:** With probability ϵ, the agent selects a random action to explore new strategies.
- **Exploitation:** With probability $1-\epsilon$, the agent selects the action with the highest estimated Q-value.

The value of ϵ follows a predefined decay schedule:

- **Training:** ϵ starts at a high value to encourage exploration and decays linearly over a fixed number of episodes, gradually shifting the agent toward exploitation.
- **Evaluation:** During testing, ϵ is set to zero, resulting in a purely greedy policy that always selects the best-known action.

5.4 Learning Algorithm

After each interaction step with the environment, the agent updates its Q-table using the standard **Bellman update equation**:

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') \right]$$

where α is the learning rate and γ is the discount factor.

5.5 Challenges and Solutions

Problem 1: Curse of Dimensionality

- **Challenge:** Pixel observations are too large to store in a Q-table.
- **Solution:** Tile coding discretizes the observation space into a finite number of states.

Problem 2: Continuous Action Space

- **Challenge:** Q-learning requires discrete actions, but the environment uses continuous controls.
- **Solution:** A discrete action wrapper maps continuous control vectors to a finite action set.

Problem 3: Exploration vs. Exploitation

- **Challenge:** The agent must explore without getting stuck in poor local optima.
- **Solution:** Linear epsilon decay encourages early exploration and later exploitation.

5.6 Results and Visual Analysis (Q-Learning)

Robustness is evaluated by comparing mean reward across multiple random seeds and multiple runs. Figures 1–4 show high variance across seeds, including occasional catastrophic drops, indicating limited stability under stochastic dynamics. Figure 5 summarizes the variability across runs.

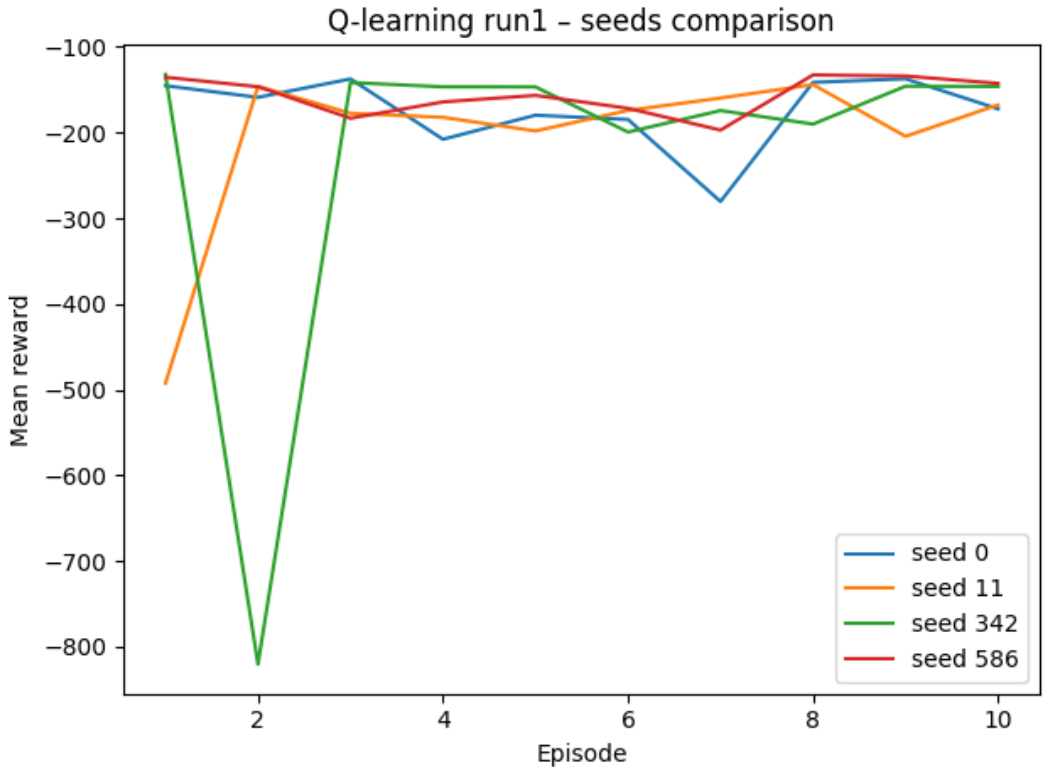


Figure 1: Q-learning (run 1): mean reward comparison across seeds.

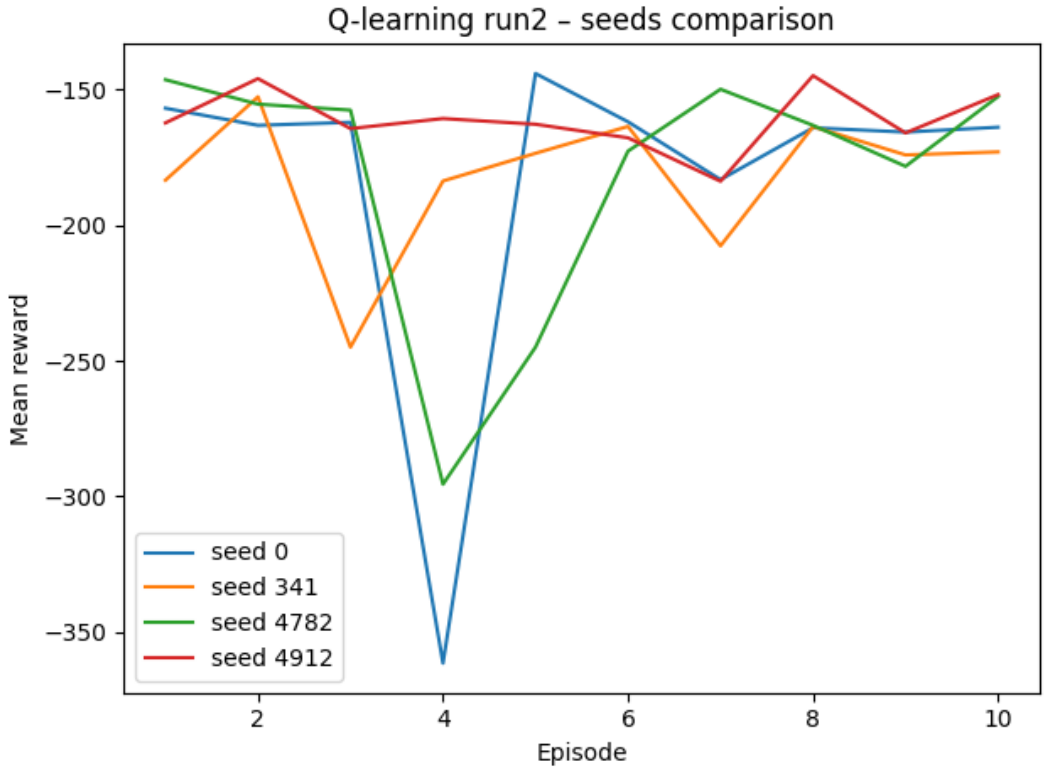


Figure 2: Q-learning (run 2): mean reward comparison across seeds.

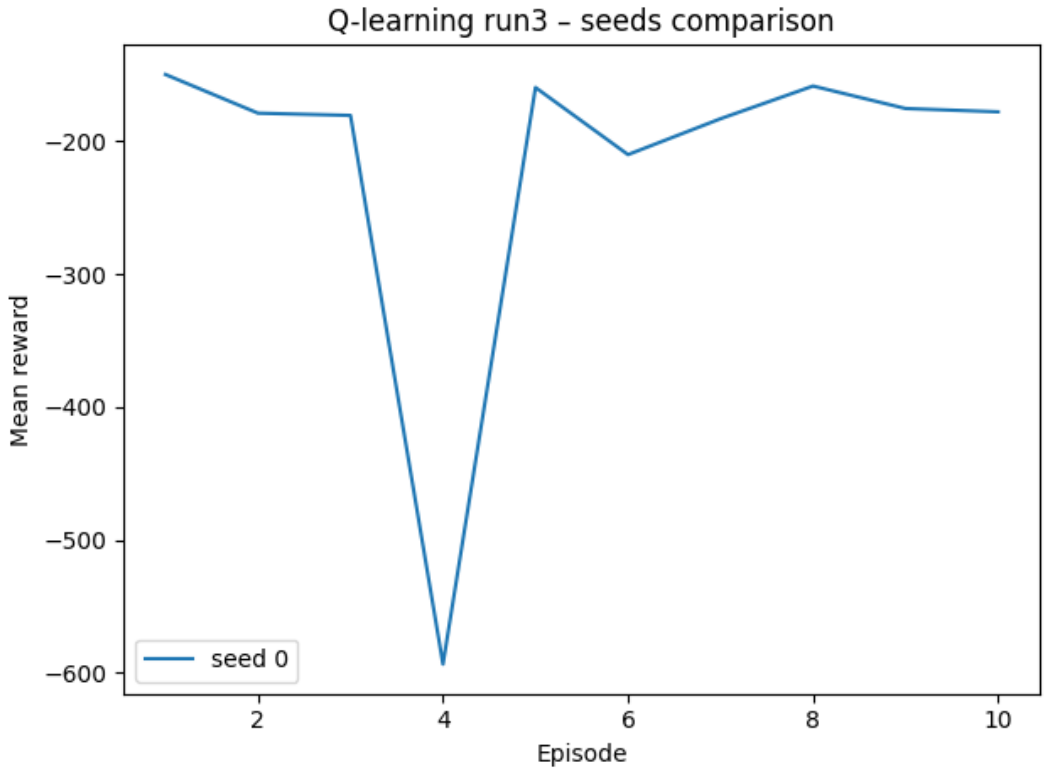


Figure 3: Q-learning (run 3): mean reward comparison across seeds.

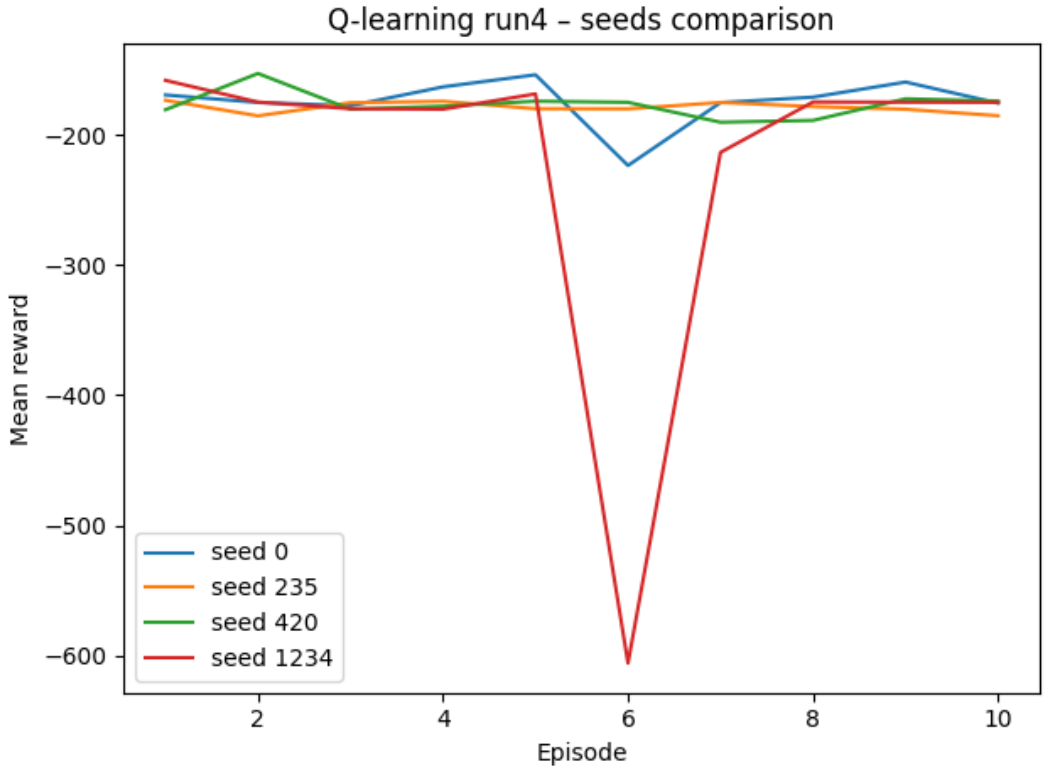


Figure 4: Q-learning (run 4): mean reward comparison across seeds.

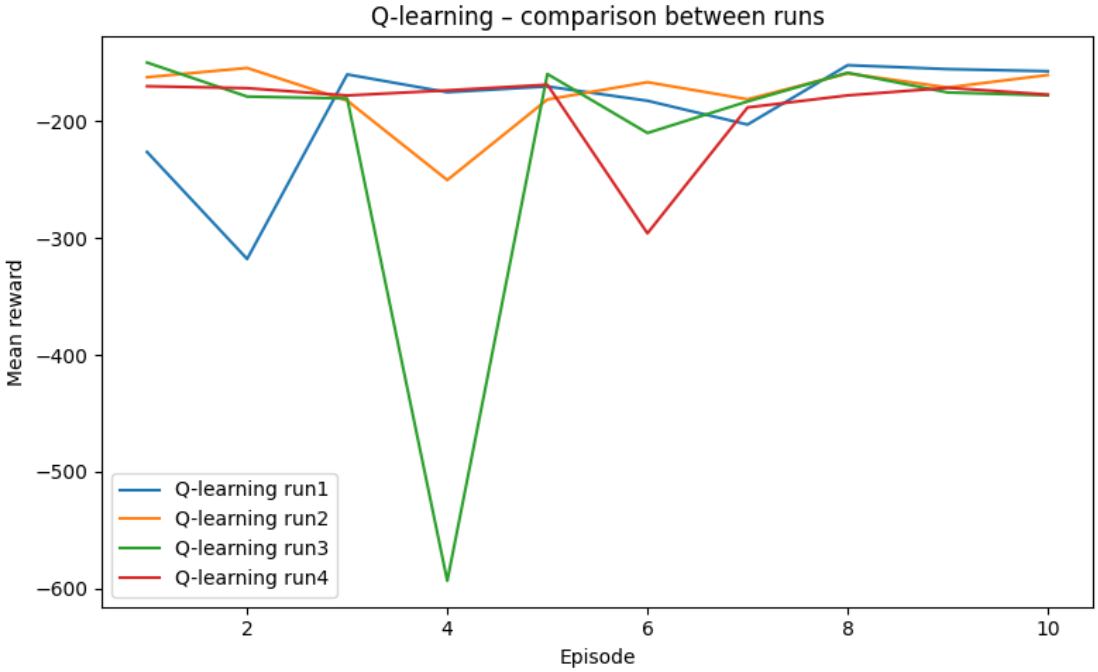


Figure 5: Q-learning: comparison between runs (mean reward per episode).

6 Agent 2: PPO (Proximal Policy Optimization)

Proximal Policy Optimization (PPO) is a policy-based reinforcement learning algorithm that learns to select optimal actions through trial and error, with the objective of maximizing long-term cumulative reward.

6.1 Agent Workflow

- Interacts with the environment and collects (o, a, r, o') transitions over a fixed number of timesteps.
- Computes the **advantage** of actions relative to the current policy baseline.
- Updates the policy to increase the probability of advantageous actions.
- Uses a **clipping mechanism** to prevent overly large policy updates.

6.2 Training Evolution

Table 1 summarizes performance as a function of training timesteps.

Table 1: PPO performance as a function of training timesteps (mean over 10 episodes, seed 0).

Training Steps	Mean Reward	Std. Deviation	Mean Off-track	Observations
10,000 timesteps	-191.52	104.97	0.399	Vehicle leaves the track quickly, exhibits abrupt movements, and fails to maintain direction.
500,000 timesteps	-63.60	163.81	0.236	Significant improvement in mean reward, but higher variance indicates inconsistent behavior across episodes.

6.3 Visual Observations and Run Comparison

Figures 6 and 7 compare two PPO configurations (run2 vs run3) for specific seeds and illustrate the sensitivity of PPO to both hyperparameters and randomness.

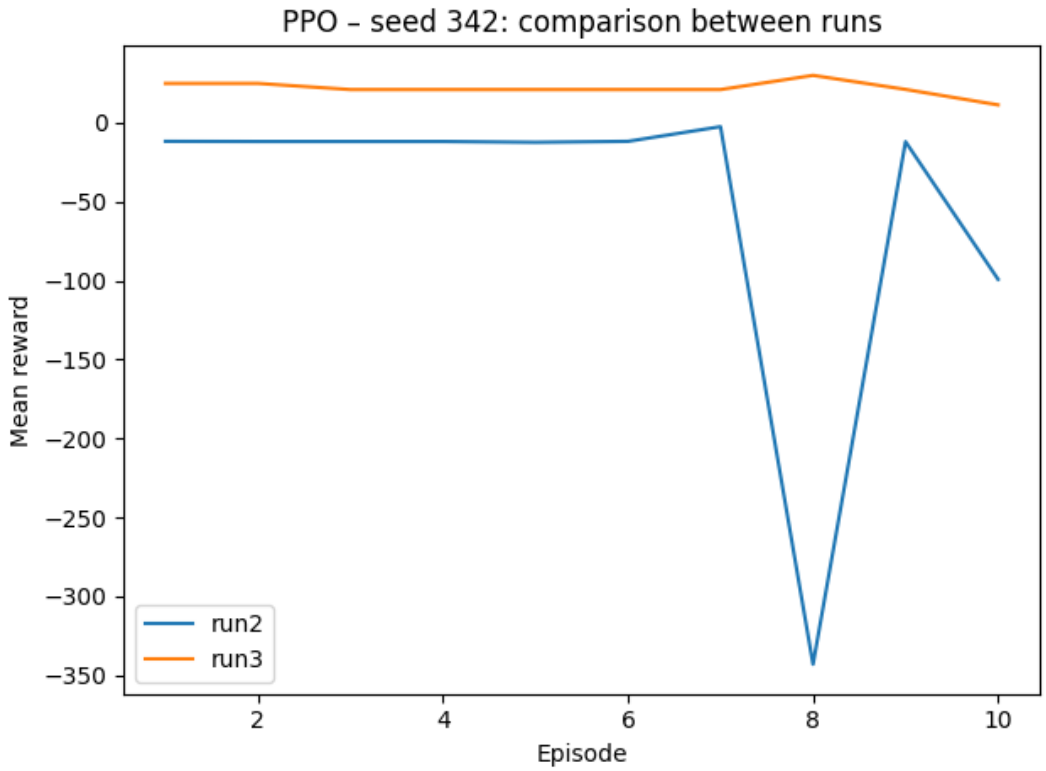


Figure 6: PPO seed 342: comparison between run2 and run3 across evaluation episodes.

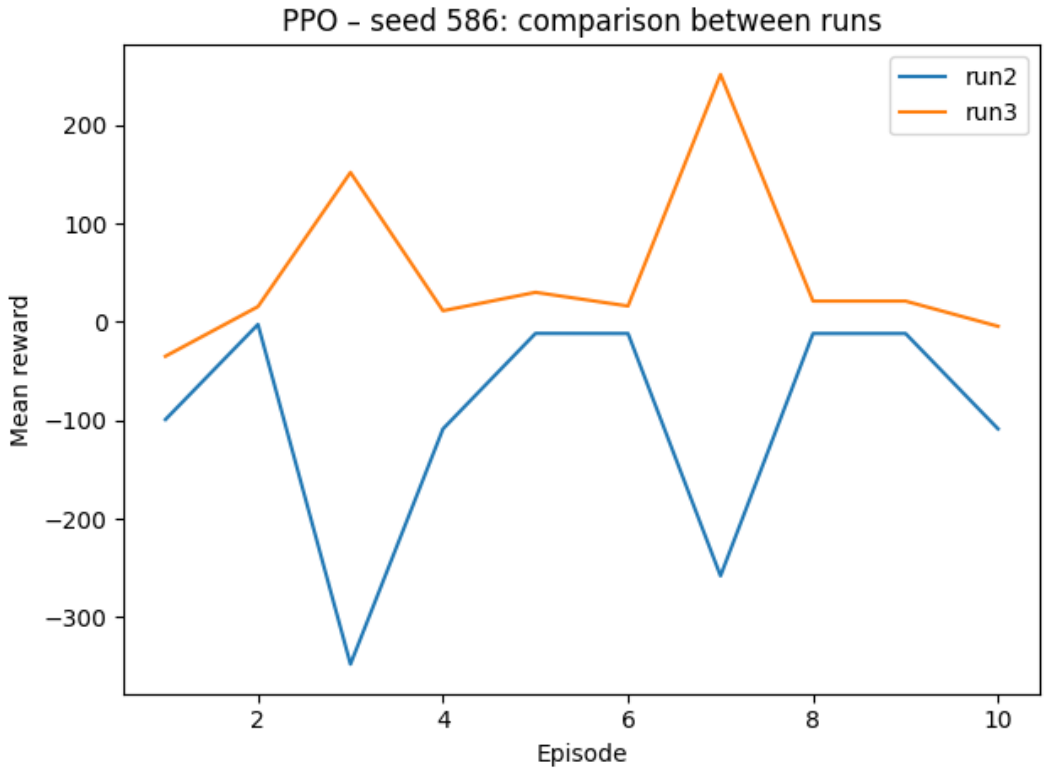


Figure 7: PPO seed 586: comparison between run2 and run3 across evaluation episodes.

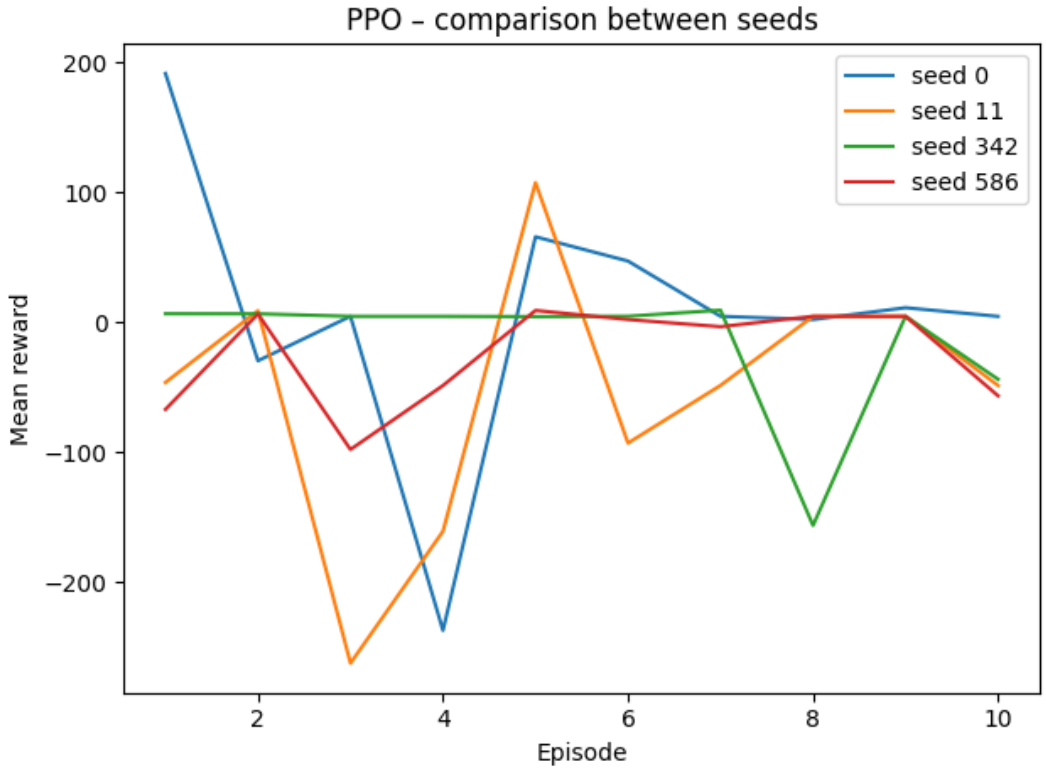


Figure 8: Final ppo comparason on all seeds

6.4 Hyperparameter Analysis

- **Entropy coefficient (`ent_coef`):** higher values increase exploration but may slow convergence.
- **Learning rate (`learning_rate`):** smaller values improve stability but slow learning.

Table 2: Impact of PPO hyperparameters at 500,000 timesteps.

Hyperparameters	Mean Reward	Std. Deviation	Mean Off-track	Observations
learning_rate = 0.00025 ent_coef = 0.00 (run 2)	-63.60	163.81	0.236	Low exploration leads to an overly deterministic policy early in training.
learning_rate = 0.0001 ent_coef = 0.01 (run 3)	76.63	95.38	0.16	Improved performance; entropy encourages exploration and reduces the risk of converging to a poor strategy.

7 Agent 3: Deep Q-Network (DQN)

7.1 Neural Function Approximation

DQN uses a convolutional neural network to approximate the optimal action-value function $Q(s, a)$ from stacked visual observations. Actions are selected via epsilon-greedy exploration during training and greedily during evaluation.

7.2 State Preprocessing and Input Representation

Frames are converted to grayscale, resized/cropped to 84×84, and stacked with $n = 4$ frames:

$$(4, 84, 84).$$

Effect of frame stacking. Using four stacked frames provides short-term temporal context, allowing the network to infer velocity and steering dynamics implicitly. With fewer frames (e.g., 1–2), the agent lacks temporal information and cornering becomes unstable; with more frames (e.g., 8), computation increases and convergence is slower with limited gains.

7.3 Action Space

The continuous control space is discretized into $n_{\text{actions}} = 5$ actions via a wrapper.

7.4 Network Architecture

A CNN with two convolutional layers followed by fully connected layers outputs Q-values for each action.

7.5 Experience Replay and Target Network

Experience replay reduces sample correlation, while a target network updated every C steps stabilizes TD targets.

7.6 Optimization Objective

The agent minimizes the TD error with an MSE loss and Adam optimizer.

7.7 Results and Visual Analysis (DQN)

Figures 9 and 10 show the DQN learning dynamics. Learning curves differ across seeds in early training, but converge towards high evaluation rewards as training progresses. The mean±std plot summarizes performance and variability across seeds.

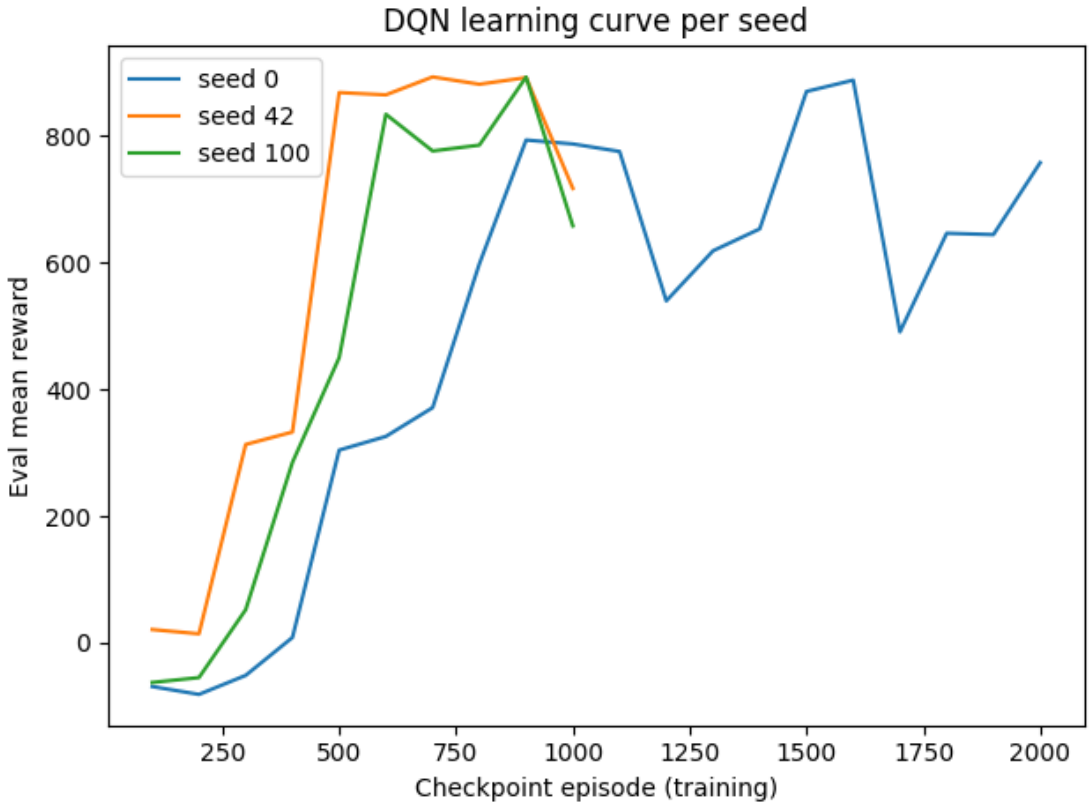


Figure 9: DQN learning curves per seed (evaluation mean reward vs checkpoint episode).

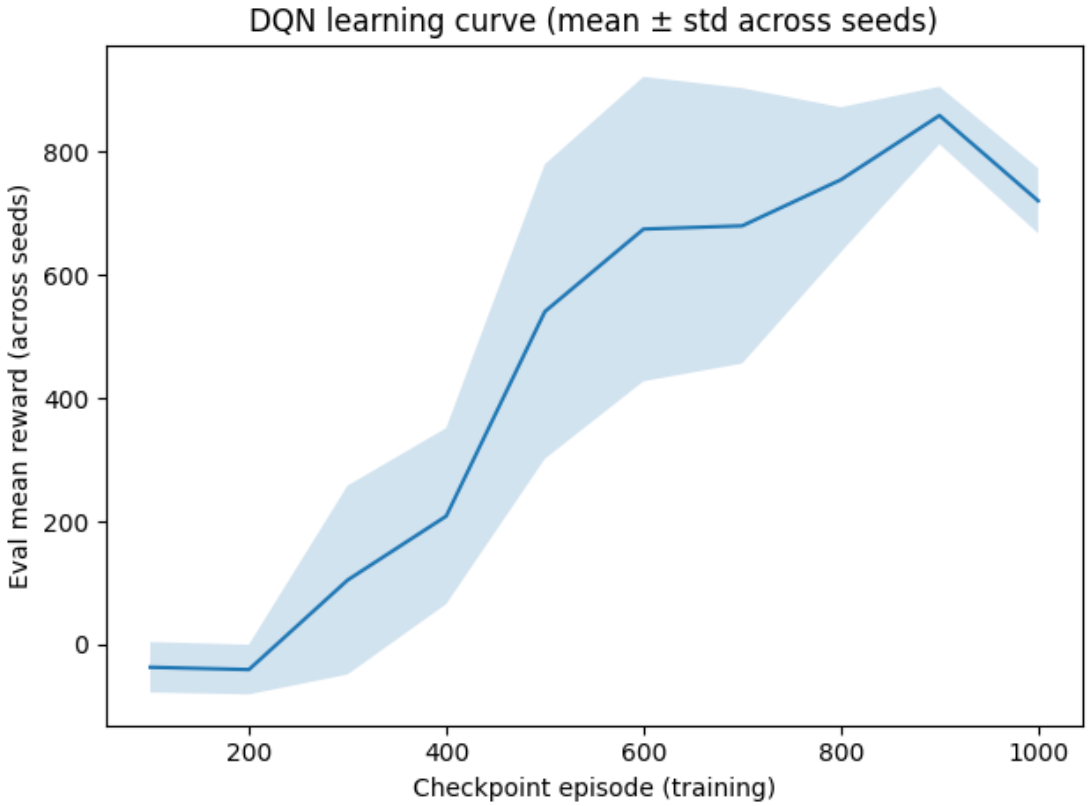


Figure 10: DQN learning curve: mean ± std across seeds (evaluation mean reward).

7.8 Other Observations and Discussion (DQN)

- **Cornering behavior:** the agent learns to position the car optimally before entering corners, minimizing braking and maintaining higher speed.
- **Failure modes:** shallow turns at high speed can cause off-track events in some seeds, suggesting a pace vs safety trade-off.
- **Recovery anomalies:** in rare cases, the agent attempts suboptimal recovery maneuvers, including turning approximately 180° and driving in the wrong direction despite penalties.

Exploration schedule and hyperparameter choices. Exploration scheduling proved critical. Faster epsilon decay led to early stagnation (around episode 200), where the agent prematurely committed to suboptimal behaviors. Slower decay enabled prolonged exploration, leading to lap completion and stronger final policies. In addition, learning rate, target update frequency (C), replay buffer size, and batch size influenced stability and convergence speed. The final configuration was selected as a compromise between training stability, sample efficiency, and computational feasibility, and it produced the most consistent behavior across seeds.

Multiple seeds and robustness. Experiments were repeated using multiple random seeds. While overall learning trends were consistent, convergence speed and peak performance varied across seeds, reflecting the stochastic nature of deep RL.

Stability and failure modes. Although DQN converges to a high-performing policy, seed-dependent failures and transient instabilities remain observable, which is typical for value-based methods in complex visual environments.

7.9 Partial Conclusions (DQN)

DQN achieves the strongest performance and meets the project objective of completing randomized circuits efficiently. However, training is computationally expensive and sensitive to hyperparameters. Moreover, because the policy relies heavily on visual input, incorporating richer F1-inspired constraints (e.g., tire wear, pit stops, energy management, weather) would require additional state variables and significant retraining, reducing scalability for strategy-heavy extensions.

8 Experimental Methodology

All agents are trained under identical environmental conditions. Each configuration is evaluated using multiple random seeds to assess robustness under stochastic dynamics.

Metrics include:

- mean episodic reward,
- reward variance (stability),
- off-track frequency.

8.1 Aggregated Metrics Across Experiments

To complement per-seed learning curves, we summarize performance across runs using boxplots. Figures 11–13 report (i) mean reward, (ii) off-track frequency, and (iii) reward standard deviation, enabling a direct comparison of performance and stability.

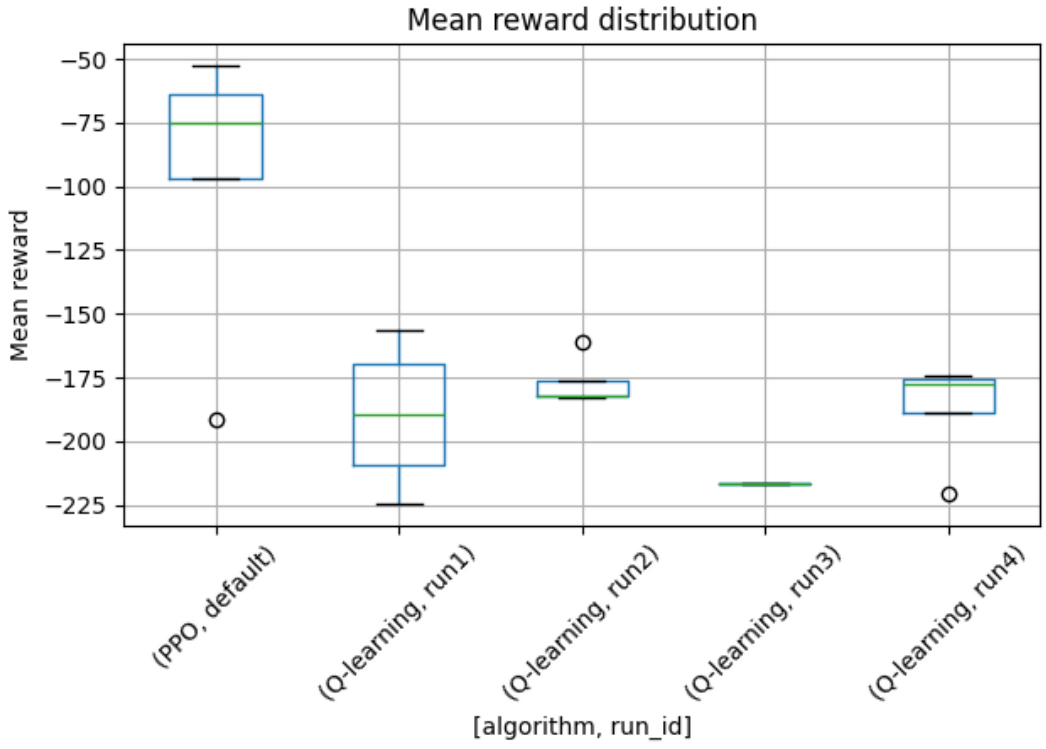


Figure 11: Mean reward distribution across algorithms and runs.

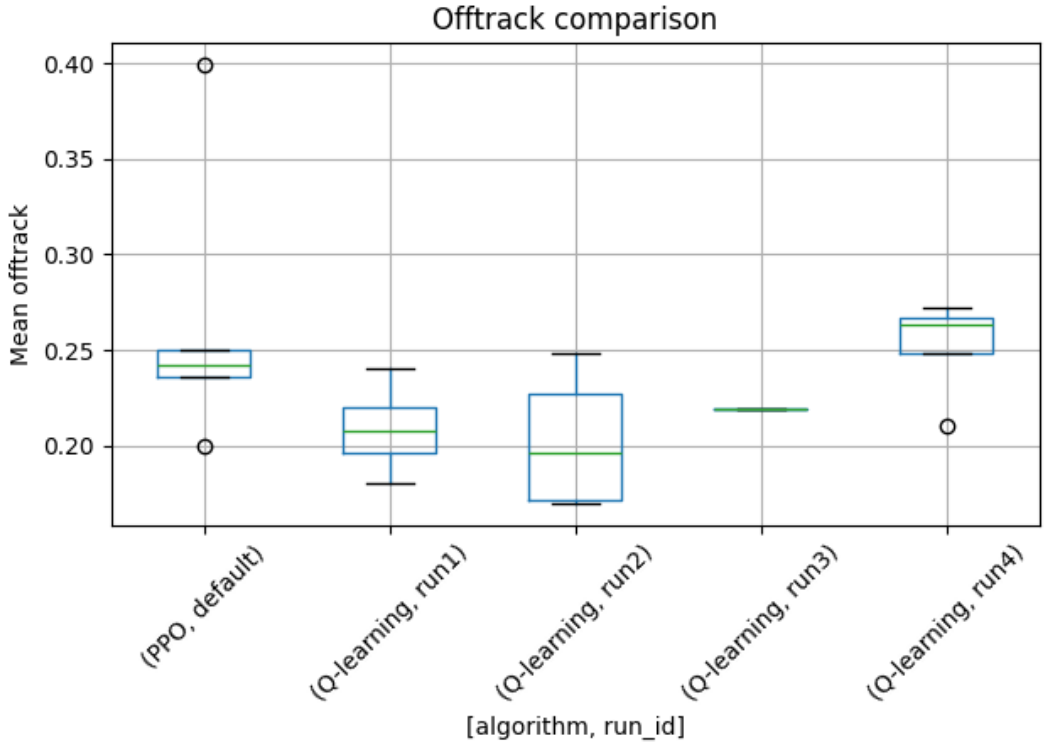


Figure 12: Off-track frequency distribution across algorithms and runs.

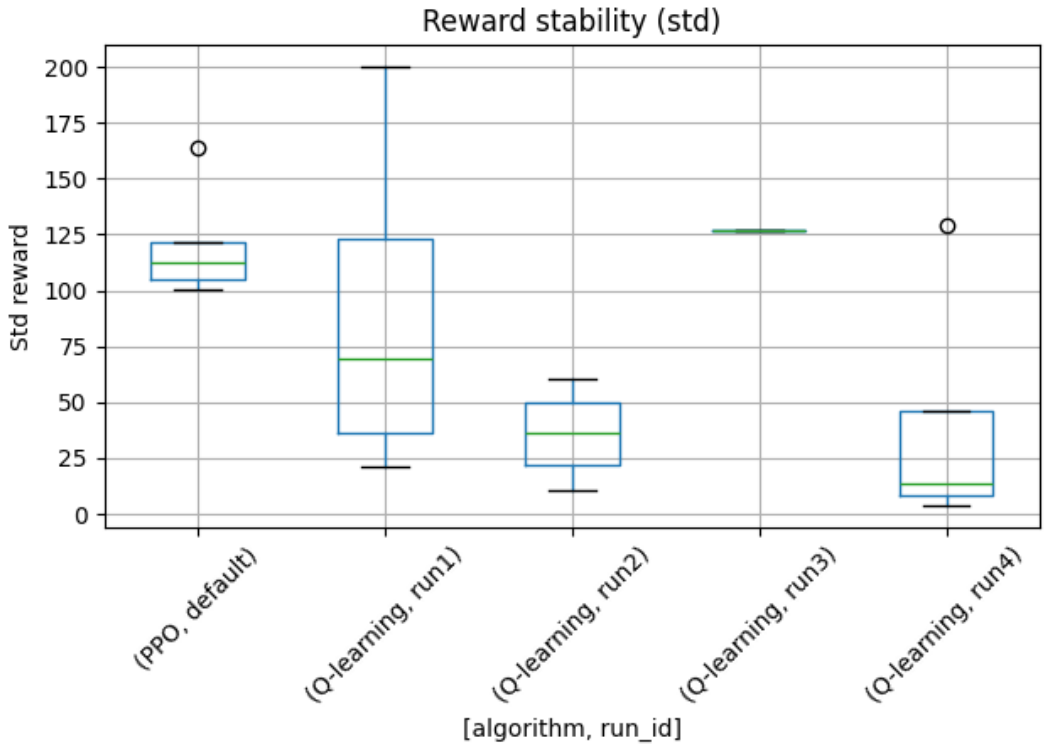


Figure 13: Reward stability (standard deviation) distribution across algorithms and runs.

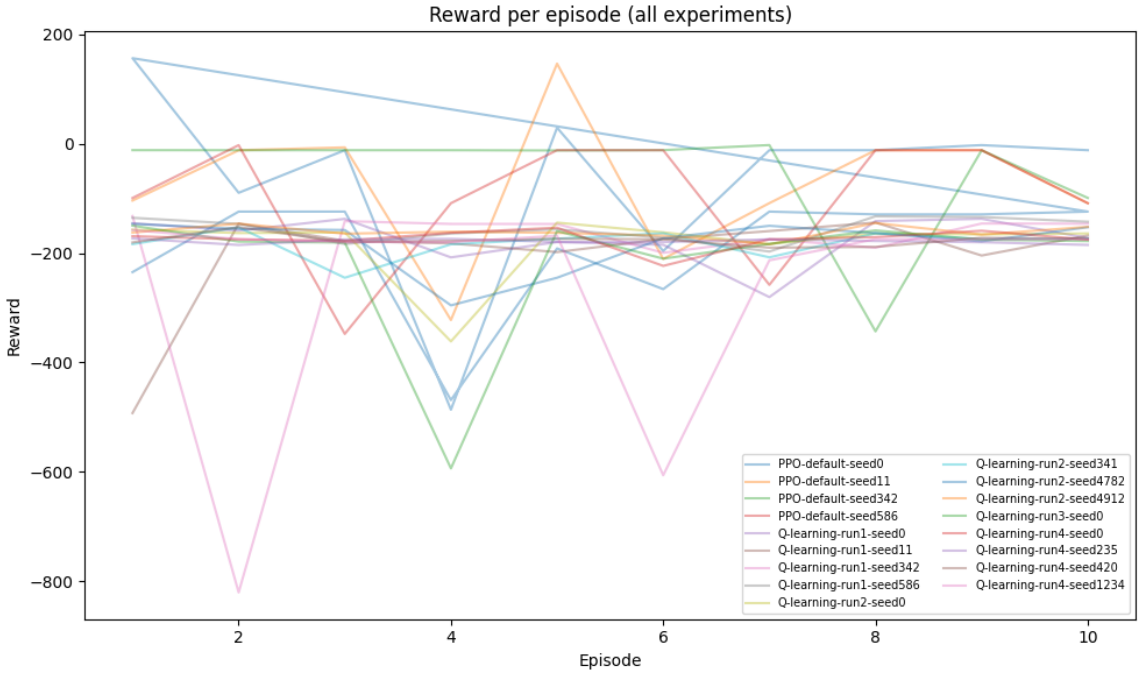


Figure 14: Reward per episode across all experiments (all runs and seeds).

9 Overall Comparison and Discussion

This section provides a comparative analysis of the three reinforcement learning approaches: tabular Q-learning, PPO, and DQN. We discuss performance, stability, computational cost, and scalability to more complex racing scenarios.

9.1 PPO vs. Q-learning (Empirical Comparison)

Figure 15 directly compares PPO and Q-learning mean rewards across evaluation episodes. PPO generally achieves higher rewards than Q-learning, while Q-learning remains consistently lower, reflecting the representational limitations of tabular methods in visual domains.

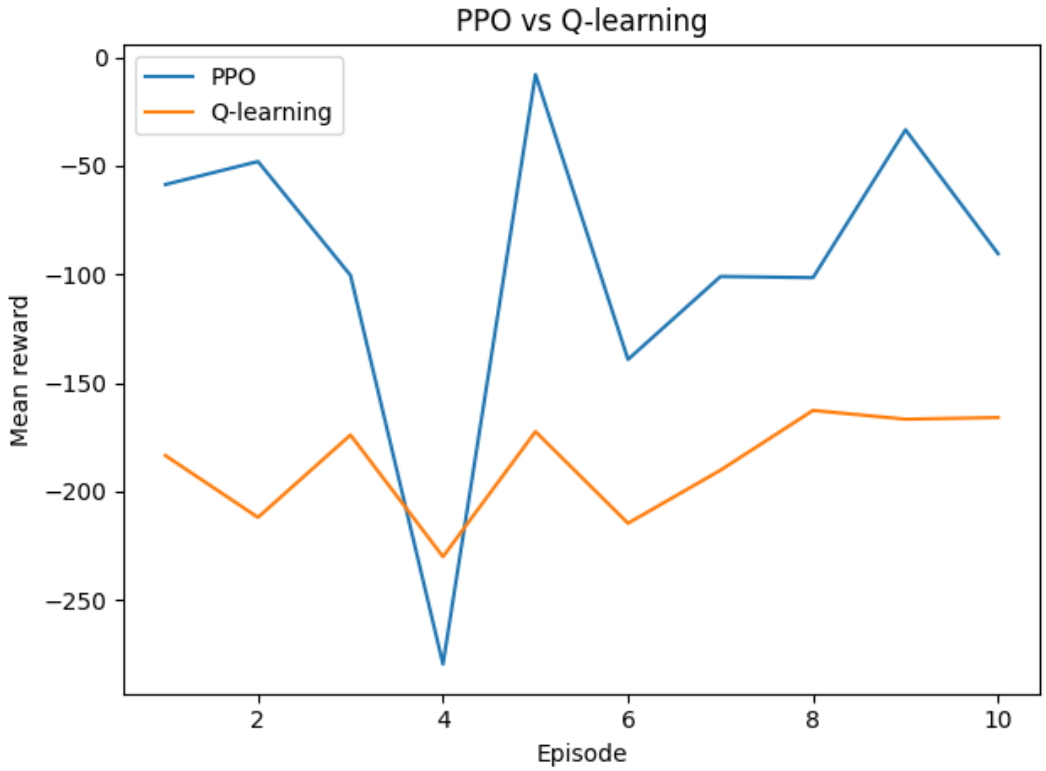


Figure 15: Direct comparison of PPO vs. Q-learning mean rewards across evaluation episodes.

9.2 Discussion of Trade-offs

- **Q-learning:** simple and interpretable, but highly unstable across seeds/runs and not scalable to image-based state spaces.
- **PPO:** naturally suited to continuous control, but sensitive to hyperparameters and prone to high variance between runs and seeds.
- **DQN:** strongest performance potential due to CNN feature extraction and stabilization mechanisms, but computationally expensive to train and sensitive to exploration scheduling.

The aggregated boxplots (Figures 11–13) support these conclusions by summarizing reward levels, off-track frequency, and stability across experiments.

10 Conclusion

This project presents a comparative study of three reinforcement learning paradigms applied to an autonomous racing task. By combining experimental evaluation with theoretical insights, we highlight the importance of robustness and stability in addition to raw performance.

The results provide a principled foundation for future work in autonomous racing and reinforcement learning-based strategy optimization.